

Prompts, System Instructions & Workflows

Project	AI Atelier: an AI-powered bathroom design platform
Primary goal	Create a bathroom concept from room dimensions, budget and aesthetic, or redesign an existing bathroom from an uploaded photograph.
Frontend	React 19 + Vite, with React Three Fiber for the interactive 3D room viewer
Backend	FastAPI (Python) with SQLAlchemy
AI services	Google Gemini (visual analysis) · Stability AI Structure Control (photo redesign) · Pollinations Flux (new-space visualisation)
Secrets policy	Real API keys live only in backend/.env, which is excluded by .gitignore. backend/.env.example carries placeholders only.

1. Overview and architecture

AI Atelier offers two journeys. **Design a New Space** turns room dimensions, a budget and an aesthetic choice into a budget-aware set of real Kohler India products and a photorealistic concept image. **Analyze My Bathroom** accepts a photo of an existing bathroom, analyses what is visibly there, and redesigns the same space while keeping its recognisable structure.

The guiding principle is a clear split of responsibilities: **AI models handle perception and imagery; deterministic code handles anything involving products, prices and budgets.** This keeps recommendations explainable and prevents a generative model from inventing SKUs or costs.

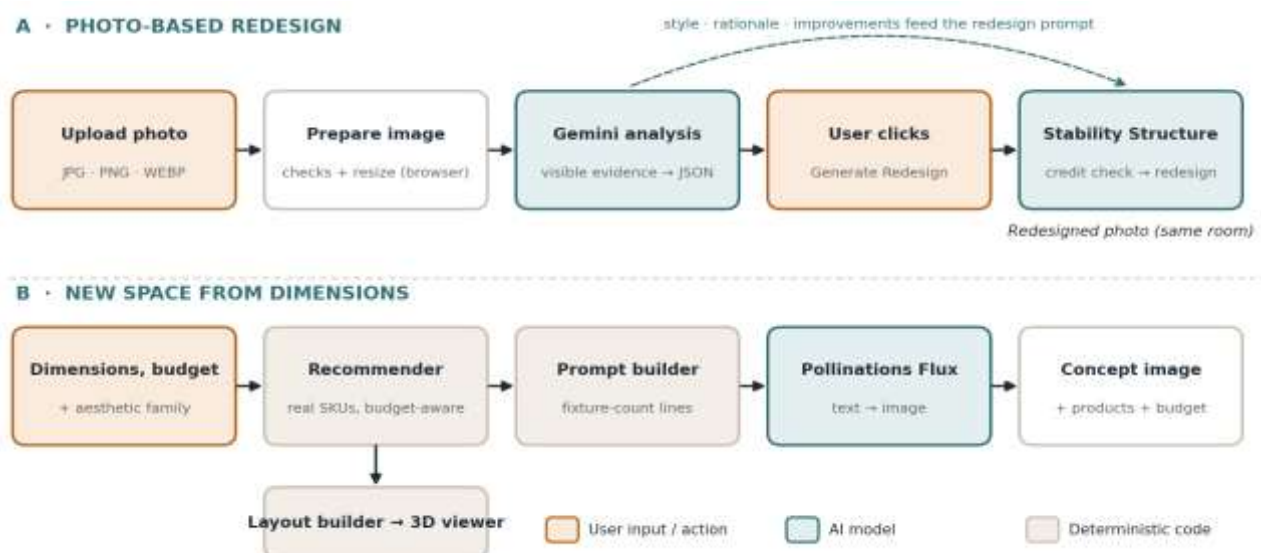


Figure 1. The two workflows and where AI models are used.

Journey	Inputs	Pipeline	Output
A. Analyze My Bathroom	One photo of the existing bathroom (JPG, PNG or WEBP)	Gemini analyses only visible evidence and returns structured JSON. After the user reviews it and clicks Generate Redesign , Stability Structure Control redesigns the same room.	Analysis panel and a redesigned photo of the same space
B. Design a New Space	Room width × depth (ft), budget (₹), aesthetic family	The recommender selects budget-aware SKUs, the prompt builder fixes fixture counts, and Pollinations Flux renders the concept. A layout builder feeds the 3D viewer.	Concept image, product list with total and remaining budget, 3D layout, budget options

2. AI components and their roles

Three external AI services and four pieces of deterministic logic make up the system. Only the first three are generative; each is given one narrow job.

Component	Type	Role	Source file
Gemini	Vision language model	Structured understanding of an uploaded photo. Returns compact JSON; never generates images.	bathroom_analyzer.py
Stability Structure Control	Image-to-image	Redesigns the uploaded photo while preserving its geometry and camera viewpoint.	stability_photo_redesign.py
Pollinations Flux	Text-to-image	Visualises a new bathroom from a detailed prompt, 1024 × 1024.	image_generator.py
Product recommender	Deterministic	Selects real Kohler India SKUs within budget, ranked by aesthetic match.	recommender.py, products.py
Budget explorer	Deterministic	Builds three trade-off tiers (Smart, Balanced, Elevated) from the same catalogue.	explorer.py
Design assistant	Rule-based	Converts short requests ("make it warmer") into budget and style settings. No LLM call.	design_assistant.py
Layout builder	Deterministic	Places fixtures for the interactive 3D viewer. Not sent to any image model.	simple_layout.py

3. Guardrails and system instructions

These rules apply across both workflows. The last column shows whether each is enforced in a prompt, in code, or by process.

Area	Rule	Enforced by
Secrets	Real API keys are never committed. .gitignore excludes .env, backend/.env, frontend/.env and generated outputs. Only placeholders are shared.	Process, .gitignore
Photo privacy	The uploaded photo travels as base64 in the request body and is held in memory for the API calls. The backend does not save uploads; only generated results are written to generated_designs/. Note that the photo is sent to Gemini and Stability for processing.	Code
Evidence-only vision	Gemini analyses only what is visible. It must not estimate room dimensions or invent hidden plumbing, construction, brands or measurements. Uncertain items are reported as "Not confidently identified". Temperature is 0.	Prompt, config

Area	Rule	Enforced by
No false brand claims	A visible fixture is never called a Kohler product unless the image clearly establishes it.	Prompt
Grounded products	Product names, SKUs and prices come only from the catalogue in products.py. Models never produce them.	Code
Fixture integrity	Prompts state exact counts for basins, toilets, showers and bathtubs, forbid duplicates, and the Stability call adds a negative prompt against duplicate fixtures.	Prompt
Structure preservation	Photo redesign keeps walls, openings, camera viewpoint and proportions, using Structure Control at strength 0.78 plus explicit prompt text.	Prompt, config
Clean imagery	No people, text, labels, prices, annotations, floor plans, collages or drawings in any output.	Prompt, negative prompt
Cost control	Paid generation starts only after an explicit user click. The backend checks the Stability balance first and blocks the call if the key is invalid or credits are insufficient.	Code
Input validation	Only JPG, PNG and WEBP. Photos must be at least 64 px per side, no wider than 2.5 : 1, and are downscaled below Stability's pixel ceiling before upload.	Code
Reliability	Retries with exponential back off, a Gemini model fallback chain, and defensive JSON parsing and normalization.	Code

4. Workflow A: photo-based redesign

The user uploads a photo of their bathroom. Analysis and generation are deliberately two separate steps so the user sees and can trust the analysis before any paid image call is made.

- 1. Upload and prepare (browser).** The frontend validates size and aspect ratio, and downscales the image if it exceeds 9,000,000 pixels (JPEG quality 0.94). Stability's limit is 9,437,184 pixels.
- 2. Analyze (Gemini).** POST /analyze-bathroom sends the image plus the analysis prompt and a strict JSON schema.
- 3. Review.** The frontend shows detected fixtures, style, materials, lighting, storage, improvements and a recommended design direction.
- 4. Generate on demand.** Only when the user clicks **Generate Redesign** does POST /generate-photo-redesign run.
- 5. Redesign (Stability).** The backend checks credits, builds the redesign prompt from the analysis, and calls Structure Control.
- 6. Display.** The redesigned image is saved and served from /images/. The source photo is hidden in the final layout but kept in memory for the API workflow.

4.1 Gemini bathroom analysis prompt

Role: structured visual understanding. The model returns compact JSON that is displayed to the user and reused to build the redesign prompt. The prompt below is sent as text alongside the image.

PROMPT · BATHROOM_ANALYZER.PY

You are the visual bathroom-analysis engine inside a Kohler bathroom design prototype.

Analyze ONLY what is visibly supported by the uploaded bathroom photograph.

Your job is not to generate an image. Your job is to understand the existing bathroom so the customer can use the analysis as a starting point for a redesign.

IMPORTANT RULES:

- Do not estimate exact room dimensions from the photograph.
- Do not invent hidden plumbing, construction conditions, products, brands, measurements, materials, accessibility conditions, defects, or fixtures that are not visibly supported.
- If something cannot be identified confidently, say "Not confidently identified".
- Keep descriptions concise and useful for an interior-design workflow.
- Do not claim that a visible fixture is a Kohler product unless the image clearly establishes that.
- "Recommended categories" means product categories that could be considered during redesign, not claims about products currently installed.

Assess:

1. visible bathroom fixtures
2. current visual style
3. visible materials and finishes
4. lighting character
5. visible storage situation
6. practical/design opportunities visible from the image
7. one recommended design direction for the redesign
8. useful product categories for the next Kohler recommendation stage

For recommended_style, prefer one of these four design families when reasonable:

- Warm Minimal
- Bold Heritage
- Wellness
- Editorial Luxe

Return JSON only with exactly these keys:

```
{
  "detected_fixtures": ["string"],
  "current_style": "string",
  "materials": ["string"],
  "lighting": "string",
  "storage": "string",
  "improvements": ["string"],
  "recommended_style": "string",
  "recommended_categories": ["string"],
  "design_rationale": "string"
}
```

Keep detected_fixtures, materials, improvements, and recommended_categories compact.
Return 3 to 5 improvements at most.

Response schema and downstream use

The backend also passes a strict responseSchema with all nine fields required, which reduces malformed output. Three fields flow into the redesign prompt.

Field	Type	Content	In redesign prompt
detected_fixtures	string[]	Visible fixtures, up to 8	No
current_style	string	Style of the existing space	No
materials	string[]	Visible materials and finishes, up to 8	No
lighting	string	Lighting character	No
storage	string	Visible storage situation	No
improvements	string[]	3 to 5 practical opportunities	Yes
recommended_style	string	One of the four design families where reasonable	Yes

Field	Type	Content	In redesign prompt
recommended_categories	string[]	Product categories worth considering, up to 8	No (reserved for recommendations)
design_rationale	string	Why this direction suits the space	Yes

Generation settings and fallback handling

- **Request:** generateContent with responseMimeType: application/json, the response schema, temperature: 0.0, maxOutputTokens: 2000, 90 s timeout.
- **Model chain:** GEMINI_MODEL (default gemini-3.6-flash), then gemini-3.5-flash, then gemini-3.5-flash-lite.
- **Retries:** up to 3 attempts per model with exponential backoff and jitter on 408, 429 and 5xx responses, and on malformed or truncated JSON. Non-retryable errors (for example an invalid key) stop immediately.
- **Parsing:** markdown fences are stripped and, if needed, the JSON object is extracted from surrounding prose.
- **Normalisation:** lists are de-duplicated and capped (improvements at 5). Missing fields fall back to safe defaults such as "Not confidently identified", and recommended_style defaults to "Warm Minimal".

4.2 Stability photo redesign prompt

Role: turn the existing photo into a redesigned concept while keeping the same recognisable space. The backend fills the prompt template with Gemini's analysis and sends it to Stability Structure Control together with the (resized) photo.

Control settings and cost protection

Endpoint	POST https://api.stability.ai/v2beta/stable-image/control/structure
Control strength	0.78: strong structure adherence with room for restyling
Output	jpeg, saved as generated_designs/stability_redesign_<timestamp>.jpg
Accepted input	image/jpeg, image/png, image/webp; at most 10 MiB after preparation
Credit preflight	GET /v1/user/balance before any paid call. Requires at least 5 credits. Blocks on an invalid key (401), an unreadable balance, or too few credits, and reports that no paid generation was started.
Retries and timeout	Up to 3 attempts with backoff on 408, 429 and 5xx. 240 s timeout.

Prompt template and negative prompt

PROMPT TEMPLATE · STABILITY_PHOTO_REDESIGN.PY

Redesign this SAME uploaded bathroom photograph into one photorealistic finished premium bathroom.

Preserve the same room structure, visible walls, openings, camera viewpoint and overall proportions. Keep it recognizably the same bathroom, but redesign it professionally.

Recommended design direction:
{recommended_style}

Design rationale:
{design_rationale}

Improvement priorities:
{improvements, one "- item" per line}

Use a Kohler-inspired premium bathroom look with better finishes, better lighting, refined fixtures, cleaner detailing, elegant vanity design and a more cohesive interior.

Important:

- preserve the existing architectural layout and viewpoint
- keep only one sink / vanity zone unless the original photo clearly has more
- keep only one toilet unless the original photo clearly has more
- keep only one shower zone unless the original photo clearly has more
- do not create duplicate sanitary fixtures
- do not create impossible plumbing
- do not turn it into a floor plan, collage, drawing or rendering board
- no text, no labels, no pricing, no annotations
- no people
- the result must be a realistic premium interior photograph

If analysis fields are missing, safe defaults apply: the style becomes "warm minimal luxury" and the improvement list becomes improve storage, improve lighting, improve material coherence.

NEGATIVE PROMPT

duplicate sink, duplicate vanity, duplicate toilet, duplicate bathtub, duplicate shower, extra sanitary fixtures, collage, floor plan, sketch, diagram, text, labels, watermark, distorted geometry, unrealistic plumbing, people

5. Workflow B: new space from dimensions

The user enters the internal room size in feet, sets a budget and picks an aesthetic. POST /generate-design then runs three steps: recommend products, build the layout for the 3D viewer, and generate the concept image. The room is passed around as a plain string such as Bathroom 10 ft width x 8 ft depth.

5.1 Aesthetic families

Each aesthetic is a short style prompt. The selected text is sent as style and is used both to rank products (by keyword match against product style tags) and as the Theme line of the image prompt.

Family	Style prompt sent to the backend
Warm Minimal	Warm minimal luxury bathroom, soft neutral stone, natural wood, sculptural fixtures, calm daylight
Bold Heritage	Bold heritage luxury bathroom, deep green tile, dark wood vanity, brass fixtures, editorial character
Wellness	Wellness sanctuary bathroom, warm sculptural architecture, spa atmosphere, soft ambient lighting
Editorial Luxe	Editorial luxury bathroom, dark accents, elegant mirrors, refined vanity, dramatic architectural lighting

5.2 Product recommender (deterministic)

This step is not a prompt. It is a rule-based selection from a catalogue of 25 Kohler India products (five each of basin, faucet, toilet, shower and bathtub, with SKU, INR price, style tags and product URL). Selected products then ground the image prompt.

- 1. Parse** the budget (must be above zero), the style keywords, and the room width, depth and area.
- 2. Select core products** in order: basin, faucet, toilet, shower. Rank candidates by number of matching style tags (highest first), then by price (lowest first). Only consider products that leave enough budget for the cheapest option in each remaining core category. If none qualify, return "Budget is too low".
- 3. Consider a bathtub** only if the room is at least 70 sq ft with both sides at least 7 ft, and the cheapest bathtub fits the remaining budget. Pick the best style match, preferring higher price on ties.
- 4. Upgrade** each non-bathtub choice to a higher-priced alternative, ranked by style match then price, whenever the remaining budget covers the difference.
- 5. Return** the product list, total cost, remaining budget, room area and whether a bathtub was added.

5.3 Pollinations image prompt

Role: generate a photorealistic new-bathroom concept. The prompt is built from the room, budget, theme, the selected real products and fixture-count rules, then sent as GET

`https://gen.pollinations.ai/image/{prompt}?model=flux&width=1024&height=1024`. Results are saved as `design_<n>.png`.

PROMPT TEMPLATE · IMAGE_GENERATOR.PY

Create ONE photorealistic 3D luxury bathroom interior.

Room: {room}
Budget: ₹{budget}
Theme: {theme}

Selected Kohler products:
{product_text}

Create a realistic 3D architectural bathroom with strong depth, correct scale, visible floor, and at least 2 intersecting walls.
Use a diagonal three-quarter camera perspective.
Do not create a flat front view or place every fixture on one wall.

STRICT FIXTURE COUNT:
{wash-station line}
{toilet line}
{shower line}
{bathtub line}

ABSOLUTELY NO DUPLICATE FIXTURES.
Never create a second basin, vanity, faucet, toilet, shower, or bathtub.
Each selected sanitary product must appear exactly once.

Make the bathroom luxurious, spacious, professionally designed, and Kohler-inspired.
Add tasteful non-plumbing decor such as mirrors, warm lighting, towels, plants, rug, shelves, and accessories.

High-end 3D architectural visualization with realistic stone, ceramic, glass, metal, wood, reflections, shadows, and natural lighting.
Magazine-quality luxury bathroom photography.

No text, labels, dimensions, arrows, prices, product names, people, or annotations.

How the fixture-count lines are built

`product_text` lists each selected product as - CATEGORY: name. The four count lines depend on which categories are present in the selection:

Category in selection	Line when present	Line when absent
basin (with faucet)	Exactly ONE wash station: ONE vanity + ONE basin + ONE faucet together.	NO basin, vanity, or faucet.
toilet	Exactly ONE toilet.	NO toilet.
shower	Exactly ONE shower zone.	NO shower.
bathtub	Exactly ONE bathtub.	NO bathtub.

Because the recommender always returns the four core categories, the bathtub line is the one that varies, driven by room size and budget.

Design history

Earlier iterations explored geometry-referenced generation, sending a top-down constraint map and later a controlled 3D reference image (one placeholder per fixture) to an image-edit endpoint. The helper functions `create_controlled_3d_reference` and `call_image_edit` remain in `image_generator.py` for reference, but the current build generates from the text prompt above. Keeping this path is useful if stricter layout control is needed later.

5.4 Layout builder, budget explorer and design assistant

Layout builder. A deterministic function positions fixtures for the interactive 3D viewer: basin back-left, shower back-right, toilet front-right, bathtub front-left. Footprints (ft) are basin 3.0 × 1.8, toilet 2.4 × 3.0, shower 3.2 × 3.2 and bathtub 5.5 × 2.8, scaled to fit smaller rooms. The layout is not passed to Pollinations.

Budget explorer. POST /explore builds three genuinely different product sets from the real catalogue without calling any image model: **Smart** (75% of budget), **Balanced** (100%) and **Elevated** (125%), each drawn from a different cost band so a higher budget really changes the products. The base budget is floored at ₹1,00,000.

Design assistant. POST /design-assistant turns a short request into proposed settings using keyword rules. It never calls an image model, costs no credits, and shows the proposal for review before anything is regenerated.

Request contains	Effect
An amount ("5 lakh", "₹450000")	Sets the budget to that amount (minimum ₹1,00,000)
cheaper, lower budget, save money	Reduces the budget by about 15%
increase budget, spend more	Increases the budget by about 15%
warm, cozy	Adds warm layered lighting, natural wood tones, soft neutral stone
luxury, premium	Adds premium detailing with refined stone and metal finishes
minimal, cleaner	Adds minimalist, uncluttered, calm, clean-lined
bright, airy	Adds bright natural daylight and an airy palette
dark, moody, dramatic	Adds moody architectural lighting and deeper tones
zen, spa, wellness	Adds a spa-like atmosphere with tactile natural materials

6. API reference

The FastAPI backend allows requests from <http://localhost:5173> and <http://127.0.0.1:5173>. Generated images are served from /images/<filename>. The analysis and redesign endpoints report failures as success: false with a message, so the UI can show a readable error.

Endpoint	Request body	Response	Logic
GET /	None	Health message	No AI
POST /analyze-bathroom	image_base64, mime_type	success, analysis (nine fields)	Gemini
POST /generate-photo-redesign	image_base64, mime_type, recommended_style, design_rationale, improvements[]	success, image (filename), engine: stability	Stability Structure Control
POST /generate-design	room, style, budget	success, image, products[], total_product_cost, remaining_budget, layout	Recommender, layout, Pollinations
POST /explore	room, style, budget	success, options[] (Smart, Balanced, Elevated)	Deterministic
POST /design-assistant	message, room, style, budget	success, budget, style, changed, message, room	Rule-based

7. Configuration and running locally

Variable	Required	Purpose
GEMINI_API_KEY	For photo analysis	Authenticates the Gemini analysis call.

Variable	Required	Purpose
GEMINI_MODEL	Optional	Primary Gemini model. Defaults to gemini-3.6-flash; fallbacks are automatic.
STABILITY_API_KEY	For photo redesign	Authenticates the balance check and the Structure Control call.
POLLINATIONS_API_KEY	For new-space design	Authenticates the Flux image request.
DATABASE_URL	Yes, at startup	SQLAlchemy connection URL. The backend creates its tables on start.

PLACEHOLDER TEMPLATE FOR BACKEND/.ENV.EXAMPLE

```
# backend/.env.example -- copy to backend/.env and fill in real values (never commit .env)
GEMINI_API_KEY=your_gemini_api_key_here
GEMINI_MODEL=gemini-3.6-flash
STABILITY_API_KEY=your_stability_api_key_here
POLLINATIONS_API_KEY=your_pollinations_api_key_here
DATABASE_URL=sqlite:///./ai_atelier.db
```

Backend: from backend/, install fastapi, uvicorn, requests, python-dotenv, pillow and sqlalchemy, then run uvicorn main:app --reload --port 8000. **Frontend:** from frontend/, run npm install then npm run dev. The frontend calls the API at http://127.0.0.1:8000.

8. Known limitations and next steps

- **Concept imagery, not drawings.** Fixture counts are controlled through prompts, so an occasional duplicate or odd detail can still appear. Outputs should be inspected before they are shared.
- **Dimensions guide products, not pixels.** In the new-space path, room size influences product choice (for example bathtub eligibility) and the prompt text, but the generated image does not follow an exact floor plan.
- **Photo redesign is Kohler-inspired.** Redesigned fixtures are not yet matched to catalogue SKUs. recommended_categories is reserved for that next stage.
- **Small catalogue.** 25 products across five categories. Growing and re-verifying it against Kohler India would widen the range of budget tiers.
- **Housekeeping.** Add a requirements.txt, automated tests for the recommender and explorer, and prompt versioning so changes to prompt text can be compared over time.

Appendix A. Sample outputs

Curated outputs from the sample_outputs/ folder. The first row shows the new-space workflow across different aesthetics; the second shows the photo-based redesign.



New space: light marble, warm timber



New space: dark stone, freestanding tub



New space: deep green tile, brass



New space : minimal with wooden elements

Photo-based redesign

Output of Stability Structure Control from an uploaded bathroom photo, guided by Gemini's analysis and the redesign prompt by Stability AI Section 4.2. *The source photograph is intentionally not included in the repository.*



Original image



Redesigned bathroom